

# Entrevista Técnica — Cinema TelaGrande (desenhe no draw.io)

---

**Como funciona este exercício.** Abaixo está uma **conversa** entre a equipe e o cliente do sistema **Cinema TelaGrande** (dono de um cinema). **Leia com atenção** e, a partir do que as pessoas dizem,

**desenhe um diagrama de classes** no [draw.io](https://draw.io) (ou [app.diagrams.net](https://app.diagrams.net)).

Não é para programar — é para **modelar**.

 **Seu diagrama precisa mostrar os quatro pilares da OO e os três tipos de relacionamento** (associação, agregação e composição). No fim há a **lista do que entregar**, um **lembrete da notação** e um **gabarito** (para o professor).

---

## A entrevista

**Gerente:** Pessoal, vamos organizar o sistema da **Cinema TelaGrande**. Vou trazer o cliente pra ele explicar, do jeito dele, o que o sistema precisa ter. Anotem tudo e depois a gente transforma em diagrama.

**Cliente:** No sistema **Cinema TelaGrande**, a peça central é **Ingresso**. Tem dois tipos: o **IngressoInteira** (o comum) e o **IngressoMeia**, que **custa metade e exige comprovação**. No fundo, os dois são um tipo de **Ingresso** — todos têm assento e preço e compartilham a mesma base.

**Dev sênior:** Entendi. Então tem uma parte **igual pra todos** e uma parte **específica** de cada tipo. Guardem isso — cheira a uma base comum com dois tipos que herdam dela.

**Cliente:** Isso. E o comportamento muda por tipo: olhando o **calcularPreco()** de cada um, pro **IngressoInteira** preço inteiro; já pro **IngressoMeia** metade do preço (com comprovação). A **mesma** operação, **resposta diferente conforme o tipo**.

**Analista de qualidade:** Um ponto sério: o preço do ingresso não pode ser mexido por fora (nunca negativo). Ninguém pode mexer nesses dados **por fora** — tem que ser por **operações controladas**. E tem uma regra que **NÃO pode falhar: não pode vender o mesmo assento duas vezes na mesma sessão**. Por isso o estado de **Venda** só anda por operações (aberta → paga; ou cancelada), nunca "na mão".

**Cliente:** Agora as ligações. Uma sessão tem MUITOS assentos; cada ingresso reserva UM assento de UMA sessão.

**Cliente:** Tem uma ligação de **parte-todo que nasce e morre junto: Venda ♦ Ingresso**. Cada ingresso pertence a uma venda e some junto se a venda for cancelada. (pense em **composição** — losango cheio ♦)

**Dev sênior:** Não confunda com o outro caso: **Sessao** ◇ **Assento** é só um **agrupamento**. Os assentos existem na sala por conta própria; a sessão só os agrupa. Se **Sessao** sumir, **Assento** continua existindo — isso é **agregação** (losango vazio ◇), bem diferente da composição.

**Gerente:** Fechou. Com isso dá pra desenhar o modelo. Mãos à obra.

## Dicas de leitura (pistas escondidas na conversa)

| O que apareceu na conversa                                                       | Onde isso te leva                                                       |
|----------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| "os dois são um tipo de <b>Ingresso</b> (mesma base)"                            | um tipo <b>geral</b> (base) e tipos <b>específicos</b> → <b>herança</b> |
| "a <b>mesma</b> operação <b>calcularPreco()</b> , resposta muda por tipo"        | operação <b>redefinida</b> em cada tipo → <b>polimorfismo</b>           |
| "O preço do ingresso não pode ser mexido por fora (nunca negativo)"              | dado <b>privado</b> + operações → <b>encapsulamento</b>                 |
| "não pode vender o mesmo assento duas vezes na mesma sessão"                     | <b>invariante</b> : o estado muda só por operação que valida            |
| " <b>Venda</b> ◆ <b>Ingresso</b> (nascem e somem juntos)"                        | <b>composição</b> (losango cheio ◆)                                     |
| " <b>Sessao</b> ◇ <b>Assento</b> (Assento existe sozinho)"                       | <b>agregação</b> (losango vazio ◇)                                      |
| "uma sessão tem MUITOS assentos; cada ingresso reserva UM assento de UMA sessão" | <b>associação</b> com <b>multiplicidade</b>                             |

## O que você deve entregar (no draw.io)

Um **diagrama de classes** que contenha, de forma clara:

1. **Abstração** — uma classe **base** que não é criada diretamente (ex.: **Ingresso**).
2. **Herança** — **IngressoInteira** e **IngressoMeia** ligados à base pelo triângulo vazio →▷.
3. **Encapsulamento** — atributos sensíveis como **privados** (−) e **operações públicas** (+) que os controlam (proteger o valor e o estado da **Venda**).
4. **Polimorfismo** — a operação **calcularPreco()** declarada na base e **redefinida** em cada tipo.
5. **Os três relacionamentos**, cada um com sua **multiplicidade**:
  - **Associação** (linha/seta): uma sessão tem MUITOS assentos; cada ingresso reserva UM assento de UMA sessão.
  - **Agregação** (losango **vazio** ◇): **Sessao** junta **Assentos** que existem sozinhos.
  - **Composição** (losango **cheio** ◆): **Venda** é feita de **Ingressos** que nascem e morrem com ela.

## Lembrete de notação (UML)

| Elemento | Como desenhar |
|----------|---------------|
|----------|---------------|

| Elemento          | Como desenhar                                                     |
|-------------------|-------------------------------------------------------------------|
| Classe abstrata   | nome em <i>itálico</i> + « <b>abstract</b> » (não se cria direto) |
| Operação abstrata | em <i>itálico</i> (cada subtipo redefine)                         |
| Visibilidade      | – privado · + público · # protegido                               |
| Herança           | linha com <b>triângulo vazio</b> ➞ apontando para a base          |
| Associação        | linha (com seta), com <b>multiplicidade</b> nas pontas            |
| Agregação         | losango <b>vazio</b> ◇ no lado do "todo"                          |
| Composição        | losango <b>cheio</b> ◆ no lado do "todo"                          |

Multiplicidades: **1** (exatamente um), **0..1** (zero ou um), **\*** (muitos), **1..\*** (um ou mais).

## ✓ Critério de "pronto"

- ☐ Há uma **classe base abstrata** e os dois tipos herdando dela.
- ☐ Nenhum atributo sensível está público; o valor e o estado da **Venda** são protegidos.
- ☐ A operação **calcularPreco()** aparece na base e é **redefinida** em cada tipo.
- ☐ Aparecem os **três** relacionamentos: **associação**, **agregação** (◇) e **composição** (◆).
- ☐ Você consegue **explicar** por que **Venda**→**Ingresso** é composição e **Sessao**→**Assento** é agregação.
- ☐ Todas as ligações têm **multiplicidade**.

← Briefing do projeto · 📅 Plano de evolução